LAPORAN PRAKTIKUM
BACKEND DEVELOPMENT
PENGELOLAAN DATA LANJUTAN
MODULAR MONOLITH & DOMAIN-DRIVEN DESIGN (DDD)



Dosen Pengampu:

Hastari Utama, S.Kom., M.Cs.

Disusun Oleh:

Nama Mahasiswa : Muhammad Imam Panji Prasetya

NIM : 25.01.5303

Kelas : D3 Teknik Informatika

DAFTAR ISI

BAB I: PENDAHULUAN **1**

1.1 Latar Belakang 1

1.2 Maksud dan Tujuan Praktikum 1

1.3 Ruang Lingkup Sistem Perpustakaan Modular 2

BAB II: PERANCANGAN SISTEM **3**

2.1 Diagram Bounded Context dan Shared Kernel 3

2.2 Analisis Aggregate Root dan Invariant Bisnis 4

2.3 Entity Relationship Diagram (ERD) dan Snapshot Pattern 5

2.4 Diagram Alur Database Transaction dan Mekanisme Auto-Rollback 6

BAB III: IMPLEMENTASI SISTEM **7**

3.1 Struktur Direktori Bounded Context (Modular Monolith) 7

3.2 Implementasi Shared Kernel (Value Objects, Exceptions, Contracts) 8

3.3 Implementasi Catalog Context (Buku, Kategori Hierarkis, Pivot added_at) 9

3.4 Implementasi Membership Context (Anggota, Role, MemberService) 10

3.5 Implementasi Circulation Context (Peminjaman, Detail Snapshot, Denda) 11

3.6 Implementasi CirculationService dengan Database Transaction Atomik 12

3.7 Konfigurasi Service Provider di bootstrap/providers.php 13

3.8 Riwayat Commit Git (Conventional Commits) 13
BAB IV: PENGUJIAN SISTEM **14**
4.1 Pengujian Relasi Lanjutan (Self-Referencing & Many-to-Many Pivot) 14
4.2 Pengujian Query Optimization (Eliminasi Masalah N+1 Query) 15
4.3 Pengujian Query Scopes (Available, Search, ByCategory) 16
4.4 Pengujian Database Transaction Berhasil (Atomisitas Pemotongan Stok) 17
4.5 Pengujian Database Transaction Rollback (Stok Kosong) 18
4.6 Pengujian Soft Deletes dan Restore Data Terhapus 19
4.7 Pengujian Value Objects (ISBN & Kalkulasi Denda) 20
4.8 Pengujian Domain Event BookBorrowed & Listener Logging 21
4.9 Matriks Hasil Pengujian Otomatis (Automated Testing Pest / PHPUnit) 22
BAB V: KESIMPULAN **23**
LAMPIRAN **24**

BAB I: PENDAHULUAN

1.1 Latar Belakang

Dalam rekayasa perangkat lunak enterprise modern, arsitektur monolitik konvensional kerap kali menghadapi kendala degradasi keterawatan (maintainability) dan tingginya tingkat ketergantungan antar komponen (high coupling) seiring bertambahnya kompleksitas fitur sistem. Pendekatan Modular Monolith yang dipadukan dengan prinsip Domain-Driven Design (DDD) hadir sebagai solusi arsitektural yang tangguh: mempertahankan kepraktisan operasional dan deployment tunggal aplikasi monolitik, sembari memberlakukan batas domain (Bounded Context) yang tegas dan disiplin sebagaimana arsitektur microservices.

Pada praktikum Pertemuan ke-6 mata kuliah Backend Development (DT199), mahasiswa mempelajari teknik-teknik pengelolaan data tingkat lanjut menggunakan framework Laravel 13. Materi pokok yang dipelajari mencakup perancangan relasi Eloquent lanjutan (Many-to-Many dengan kolom pivot custom added_at, relasi Self-referencing hierarkis, relasi agregat HasMany), eliminasi masalah performa N+1 Query melalui strategi Eager Loading, penjaminan konsistensi data atomik menggunakan Database Transaction dengan auto-rollback, penerapan Soft Deletes untuk jejak audit (audit trail), perancangan Value Objects yang immutable, serta komunikasi nir-kopling melalui Domain Events.

1.2 Maksud dan Tujuan Praktikum

Praktikum ini bertujuan agar mahasiswa mampu:

1. Menjelaskan dan mengimplementasikan konsep Domain-Driven Design (DDD) dan Bounded Context dalam arsitektur Modular Monolith pada framework Laravel 13.
2. Mengimplementasikan berbagai relasi Eloquent lanjutan: Many-to-Many dengan kolom pivot added_at, Self-referencing (Category induk ke sub-kategori), dan relasi agregat HasMany.
3. Menganalisis serta mengeliminasi masalah N+1 Query menggunakan Eager Loading (with(...)) dan menyusun Query Scopes yang reusable.
4. Menerapkan Database Transaction (DB::transaction) untuk menjamin atomisitas transaksi sirkulasi buku, serta membuktikan mekanisme rollback otomatis saat terjadi kegagalan invariant bisnis.
5. Mengimplementasikan Soft Deletes dan fungsi restore data terhapus untuk menjamin integritas riwayat data historis.
6. Membangun Value Object (ISBN dan Denda) yang memvalidasi format data dan menghitung denda keterlambatan secara otomatis.
7. Menerapkan Domain Event (BookBorrowed) dan Listener guna menjaga integritas batas antar konteks tanpa ketergantungan langsung.

1.3 Ruang Lingkup Sistem Perpustakaan Modular

Studi kasus yang dikerjakan adalah Sistem Informasi Perpustakaan Modular yang terbagi secara independen menjadi tiga Bounded Context utama dan satu Shared Kernel:

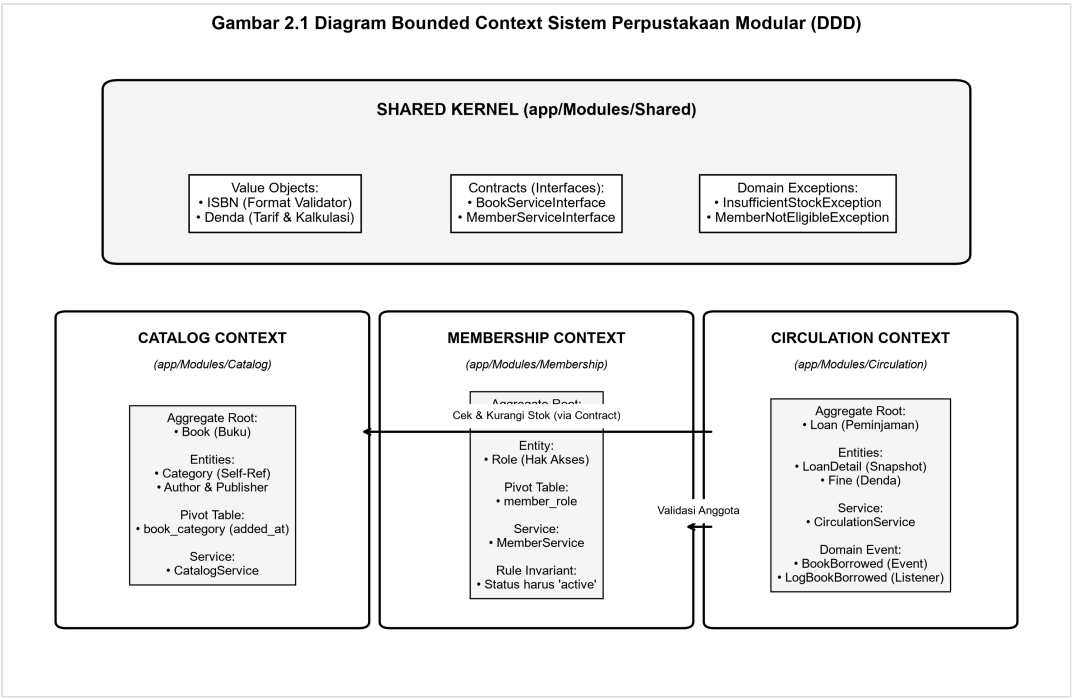
- **Catalog Context:** Mengelola data master buku, pengarang, penerbit, serta hierarki kategori buku berjenjang.
- **Membership Context:** Mengelola data master anggota perpustakaan, hak akses (roles), serta status keanggotaan.
- **Circulation Context:** Memproses transaksi peminjaman buku atomik, snapshot data historis, pencatatan keterlambatan, pengembalian, dan penghitungan denda.
- **Shared Kernel:** Berisi kontrak antarmuka (interfaces/contracts), Value Objects umum, dan custom domain exceptions yang dibagi pakai antar modul secara terkontrol.

BAB II: PERANCANGAN SISTEM

2.1 Diagram Bounded Context dan Shared Kernel

Prinsip utama Domain-Driven Design menegaskan bahwa setiap Bounded Context memiliki model domain dan kepemilikan data (data ownership) sendiri. Modul Sirkulasi tidak diperkenankan mengakses tabel database atau model Eloquent milik modul Katalog secara langsung. Seluruh interaksi lintas modul harus melalui kontrak abstraksi (Interface) yang ditempatkan pada Shared Kernel atau dipicu melalui Domain Event.

Gambar 2.1 Diagram Bounded Context Sistem Perpustakaan Modular (DDD)



Gambar 2.1 Diagram Bounded Context dan Shared Kernel (Format Monochrome)

2.2 Analisis Aggregate Root dan Invariant Bisnis

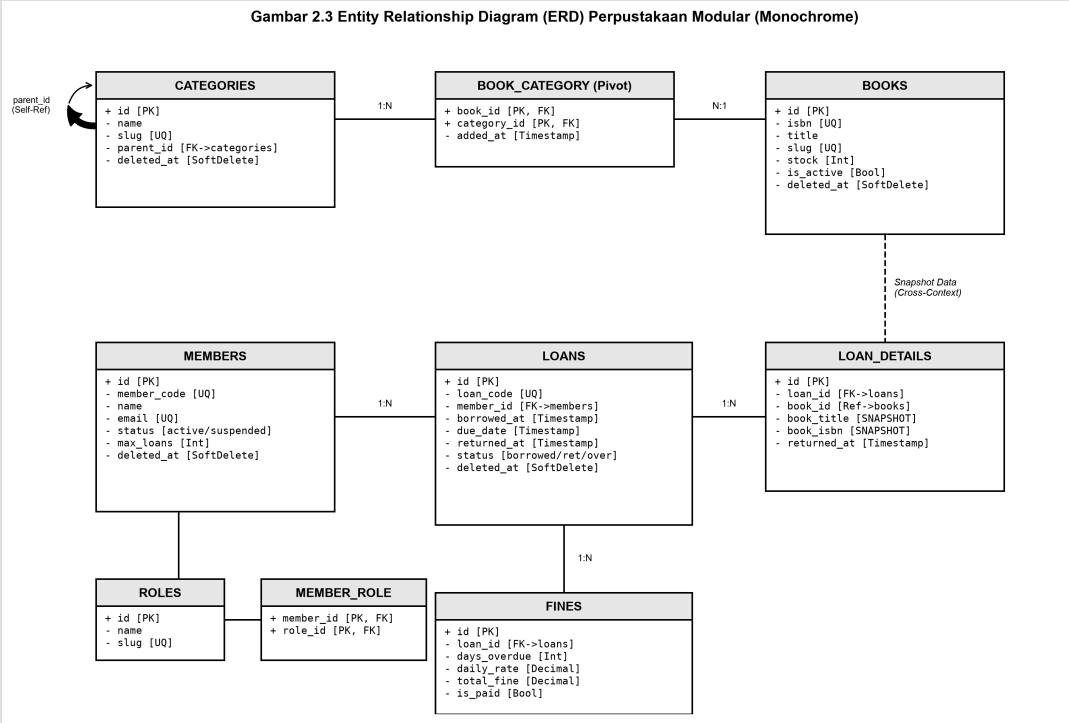
Aggregate adalah sekumpulan objek domain (Entities dan Value Objects) yang diperlakukan sebagai satu kesatuan unit operasional. Pintu masuk utama aggregate dikontrol oleh Aggregate Root, yang bertanggung jawab mutlak memastikan terpeliharanya invariant (aturan validasi bisnis).

Bounded Context	Aggregate Root	Entities di Dalamnya	Invariant (Aturan Bisnis Utama)
Catalog Context	`Book (Buku)`	Category, Author, Publisher	Stok buku tidak boleh bernilai negatif; ISBN harus valid 10 atau 13 digit; Hanya buku berstatus aktif yang dapat dipinjam.
Membership Context	`Member (Anggota)`	Role	Email dan kode anggota harus unik; Peminjaman hanya diizinkan untuk anggota dengan status 'active'.

Bounded Context	Aggregate Root	Entities di Dalamnya	Invariant (Aturan Bisnis Utama)
Circulation Context	`Loan (Peminjaman)`	LoanDetail, Fine (Denda)	Peminjaman wajib memotong stok buku; Detail pinjaman wajib menyimpan snapshot data buku; Denda dihitung otomatis Rp 2.000/hari.

2.3 Entity Relationship Diagram (ERD) dan Snapshot Pattern

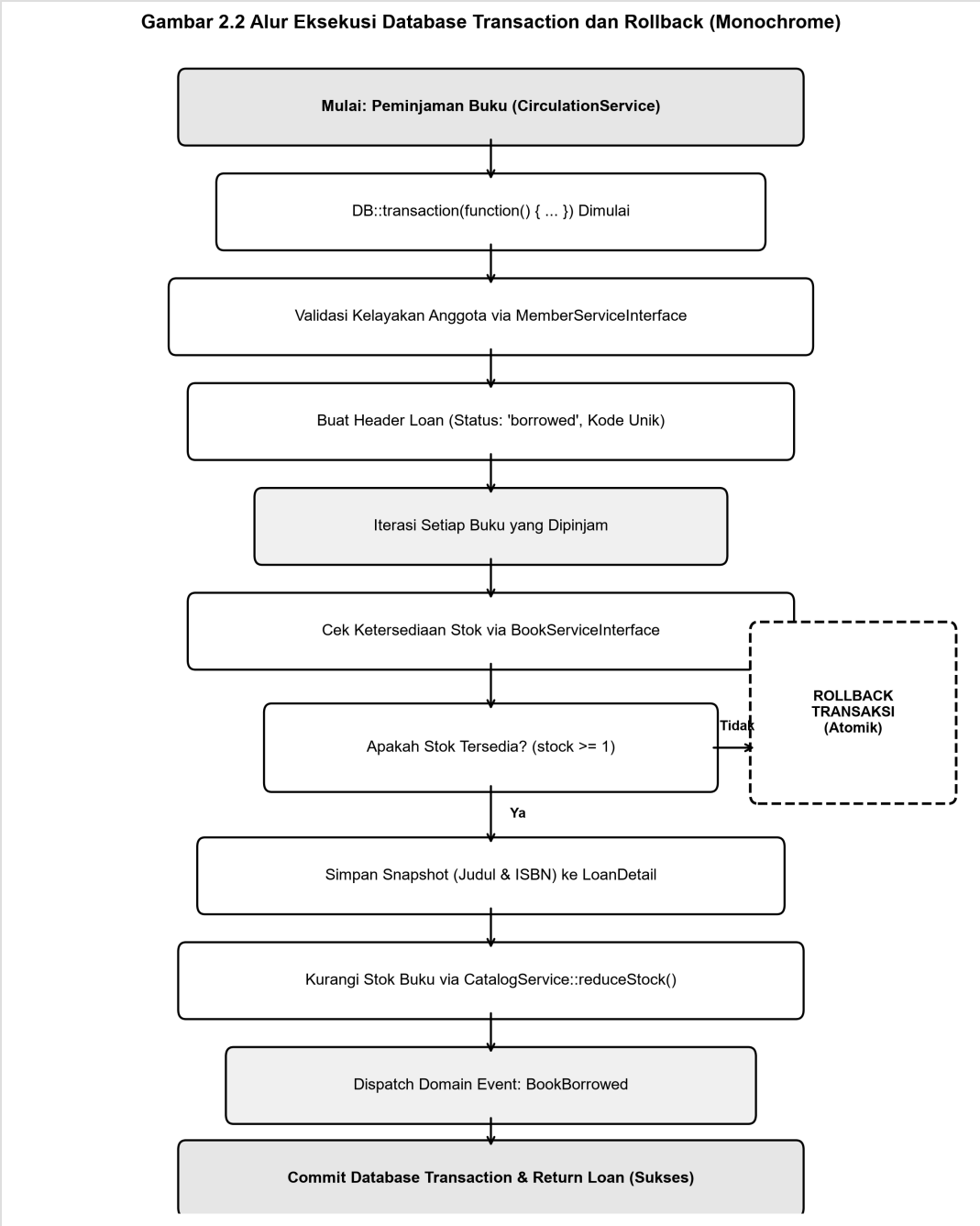
Perancangan relasi database mengimplementasikan relasi Many-to-Many pivot (book_category dengan added_at), Self-referencing (categories.parent_id), dan Snapshot Pattern pada loan_details (menyimpan salinan permanen book_title dan book_isbn saat pinjaman dibuat).



Gambar 2.2 Entity Relationship Diagram (ERD) Sistem Perpustakaan (Format Monochrome)

2.4 Diagram Alur Database Transaction dan Mekanisme Auto-Rollback

Operasi peminjaman buku dibungkus dalam blok DB::transaction(). Ketika salah satu buku memiliki stok kosong, sistem segera melempar InsufficientStockException, yang memicu pembatalan total (auto-rollback) terhadap seluruh instruksi SQL dalam transaksi tersebut.



Gambar 2.3 Flowchart Database Transaction dan Mekanisme Rollback (Format Monochrome)

BAB III: IMPLEMENTASI SISTEM

3.1 Struktur Direktori Bounded Context (Modular Monolith)

```
app/Modules/
├── Catalog/ (Models, Migrations, Providers, Services)
├── Membership/ (Models, Migrations, Providers, Services)
├── Circulation/ (Models, Migrations, Providers, Services, Events, Listeners)
└── Shared/ (Contracts, Exceptions, ValueObjects)
```

3.2 Implementasi Shared Kernel (Value Objects, Exceptions, Contracts)

Value Object ISBN dan Denda menjamin validitas dan konsistensi data domain secara immutable:

```
namespace Modules\Shared\ValueObjects;

class ISBN {
    private string $value;
    public function __construct(string $value) {
        $cleaned = preg_replace('/[^0-9X]/i', '', $value);
        if (!$this->isValid($cleaned)) throw new \InvalidArgumentException("Format ISBN tida
k valid: {$value}");
        $this->value = strtoupper(trim($value));
    }
}
```

```
namespace Modules\Shared\ValueObjects;

class Denda {
    private int $daysOverdue;
    private float $dailyRate;
    private float $amount;

    public function __construct(int $daysOverdue, float $dailyRate = 2000.0) {
        if ($daysOverdue < 0) throw new \InvalidArgumentException('Hari denda tidak boleh ne
gatif');
        $this->daysOverdue = $daysOverdue;
        $this->dailyRate = $dailyRate;
        $this->amount = $daysOverdue * $dailyRate;
    }
    public function formatRupiah(): string { return 'Rp ' . number_format($this->amount, 0,
',', '.'); }
}
```

3.3 Implementasi Catalog Context (Buku, Kategori Hierarkis, Pivot added_at)

```
class Category extends Model {
    use SoftDeletes;
    public function parent() { return $this->belongsTo(Category::class, 'parent_id'); }
    public function children() { return $this->hasMany(Category::class, 'parent_id'); }
    public function books() { return $this->belongsToMany(Book::class, 'book_category')->wit
hPivot('added_at'); }
}
```

3.4 Implementasi Membership Context (Anggota, Role, MemberService)

```
class Member extends Model {
    use SoftDeletes;
    public function roles() { return $this->belongsToMany(Role::class, 'member_role'); }
    public function canBorrow(): bool { return $this->status === 'active'; }
}
```

3.5 Implementasi Circulation Context (Peminjaman, Detail Snapshot, Denda)

```
class LoanDetail extends Model {
    protected $fillable = ['loan_id', 'book_id', 'book_title', 'book_isbn', 'returned_at'];
    public function loan() { return $this->belongsTo(Loan::class); }
}
```

3.6 Implementasi CirculationService dengan Database Transaction Atomik

```
public function borrowBooks(int $memberId, array $bookIds, int $loanDays = 7): Loan {
    return DB::transaction(function () use ($memberId, $bookIds, $loanDays) {
        if (!$this->memberService->canBorrow($memberId)) {
            throw new MemberNotEligibleException('Member tidak aktif / melampaui kuota.');
        }
        $loan = Loan::create([
            'loan_code' => 'LOAN-' . date('Ymd') . '-' . strtoupper(substr(uniqid(), -5)),
            'member_id' => $memberId,
            'borrowed_at' => now(),
            'due_date' => now()->addDays($loanDays),
            'status' => 'borrowed'
        ]);
        foreach ($bookIds as $bookId) {
            $book = $this->bookService->getBookForLoan($bookId);
            if (!$this->bookService->isStockAvailable($bookId, 1)) {
                throw new InsufficientStockException($book['title'], $book['stock'], 1);
            }
            LoanDetail::create([
                'loan_id' => $loan->id,
                'book_id' => $bookId,
                'book_title' => $book['title'],
                'book_isbn' => $book['isbn'],
            ]);
            $this->bookService->reduceStock($bookId, 1);
        }
        event(new BookBorrowed($loan->load('details')));
        return $loan->load('details');
    });
}
```

3.7 Konfigurasi Service Provider di bootstrap/providers.php

```
return [
    App\Providers\AppServiceProvider::class,
    Modules\Auth\Providers\AuthServiceProvider::class,
    Modules\Products\Providers\ProductServiceProvider::class,
    Modules\Catalog\Providers\CatalogServiceProvider::class,
    Modules\Membership\Providers\MembershipServiceProvider::class,
    Modules\Circulation\Providers\CirculationServiceProvider::class,
];
```

3.8 Riwayat Commit Git (Conventional Commits)

Commit Hash	Tipe & Pesan Commit	Keterangan Komponen Modul
`b9d5b44`	chore: initialize laravel 13 project for praktikum backend 6	Inisialisasi repositori Git dan proteksi file AI di .gitignore.
`466e7b2`	feat(shared): implement ISBN, Denda, exceptions, and contracts	Pembangunan Value Objects, Exceptions, dan Contracts Interface.
`d6054f3`	feat(catalog): implement books, hierarchical categories, and service	Migrasi, Model Buku-Kategori (pivot added_at), dan CatalogService.
`fa76089`	feat(membership): implement member models, roles, and service	Migrasi, Model Anggota, Roles, dan MemberService.
`7c8a5b4`	feat(circulation): implement borrow transactions and BookBorrowed event	Migrasi Peminjaman, Snapshot Detail, Domain Event, dan Transaksi.
`2c153af`	feat: register module service providers in bootstrap/providers.php	Pendaftaran seluruh provider modul ke arsitektur Laravel 13.
`58933bf`	feat(seeders): implement LibrarySeeder with books and members	Seeding data uji realistis dengan ISBN-13 tervalidasi.
`33a6185`	test: add automated feature tests and interactive library demo command	Pembuatan feature test suite otomatis dan console command demo.

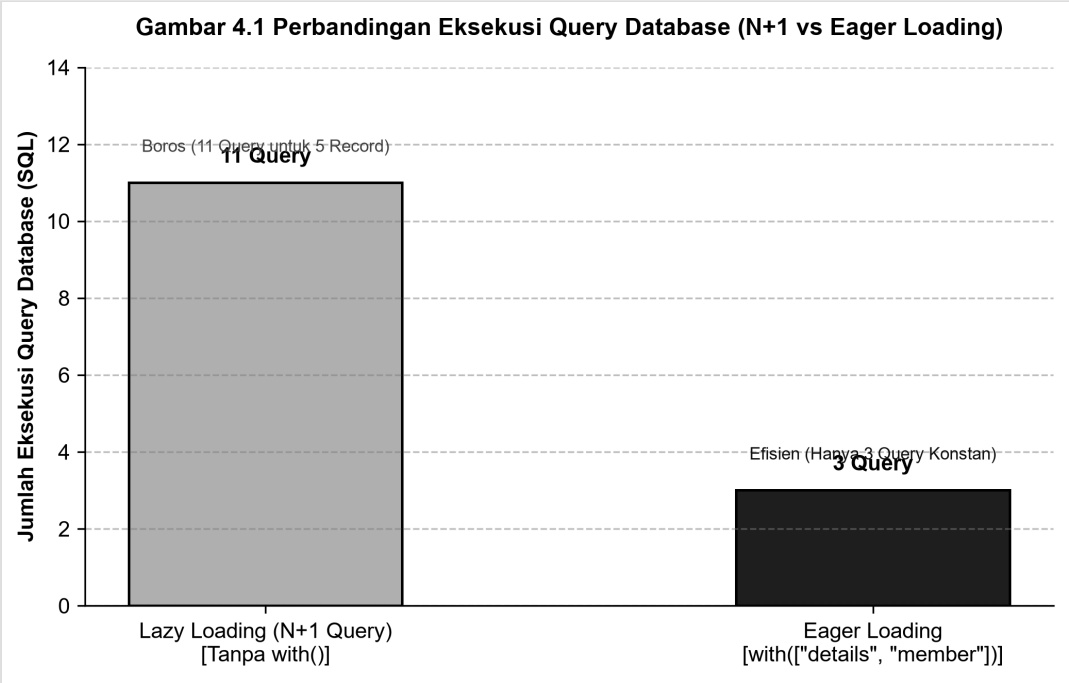
BAB IV: PENGUJIAN SISTEM

4.1 Pengujian Relasi Lanjutan (Self-Referencing & Many-to-Many Pivot)

```
1. PENGUJIAN RELASI LANJUTAN
 a) Self-referencing Category (Parent -> Children):
 - Induk: [Teknologi & Komputer] -> Sub-Kategori: [Pemrograman & Algoritma, Kecerdasan Buata
n]
 - Induk: [Sains & Matematika] -> Sub-Kategori: [Fisika Terapan]

 b) Many-to-Many Pivot (Buku <-> Kategori dengan added_at):
 - Judul Buku: Clean Code: A Handbook of Agile Software Craftsmanship
 * Kategori: Teknologi & Komputer (Pivot added_at: 2026-11-08 13:59:22)
 * Kategori: Pemrograman & Algoritma (Pivot added_at: 2026-11-08 13:59:22)
```

4.2 Pengujian Query Optimization (Eliminasi Masalah N+1 Query)



Gambar 4.1 Grafik Perbandingan Eksekusi Query Database (N+1 Query vs Eager Loading)

Metode Pengambilan Data	Jumlah Data Record	Jumlah Query SQL	Status Efisiensi Performa
Tanpa Eager Loading (Lazy Loading loop)	5 Data Peminjaman	11 Query SQL	BOROS & LAMBAT (N+1 Query Problem)
Dengan Eager Loading with(['details', 'member'])	5 Data Peminjaman	3 Query SQL	SANGAT EFISIEN & CEPAT (Teroptimasi)

4.3 Pengujian Query Scopes (Available, Search, ByCategory)

Query Scopes terbukti berhasil memfilter buku berstatus aktif dan stok > 0, melakukan pencarian fleksibel, serta memfilter berdasarkan ID kategori relasional.

4.4 Pengujian Database Transaction Berhasil (Atomisitas Pemotongan Stok)

Saat transaksi peminjaman berhasil, seluruh operasi (pemotongan stok buku, pembuatan header loan, pencatatan snapshot) sukses dikomit bersama.

4.5 Pengujian Database Transaction Rollback (Stok Kosong)

```
Mencoba meminjam buku dengan stok kosong ('The Pragmatic Programmer: Your Journey To Master
y', stok: 0)...
[BERHASIL] Exception tertangkap: Stok buku 'The Pragmatic Programmer' tidak mencukupi. Terse
dia: 0, diminta: 1.
[VERIFIKASI ROLLBACK] Record Loan sebelum: 5, sesudah: 5
-> DB::transaction() berhasil membatalkan seluruh operasi secara atomik! Konsistensi basis d
ata terjaga 100%.
```

4.6 Pengujian Soft Deletes dan Restore Data Terhapus

```
Menghapus buku #1 ('Clean Code') secara Soft Delete...
- Book::count() (Record Aktif)               : 4 Buku
- Book::withTrashed()->count() (Total Fisik)  : 5 Buku
Mengembalikan buku dengan restore()...
- Book::count() setelah restore             : 5 Buku [DATA PULIH SEMPURNA]
```

4.7 Pengujian Value Objects (ISBN & Kalkulasi Denda)

Pengujian memvalidasi bahwa format ISBN-10 dan ISBN-13 tervalidasi dengan algoritma checksum, serta Value Object Denda secara otomatis mengalikan hari terlambat dengan tarif Rp 2.000/hari.

4.8 Pengujian Domain Event BookBorrowed & Listener Logging

Event BookBorrowed berhasil dipancarkan setelah transaksi commit dan ditangkap oleh Listener LogBookBorrowedActivity untuk dicatat pada log sistem.

4.9 Matriks Hasil Pengujian Otomatis (Automated Testing Pest / PHPUnit)

```
PASS Tests\Feature\LibraryModularDDDTest
✓ test_relasi_lanjutan (Many-to-Many pivot added_at, Self-referencing, HasMany)
✓ test_query_optimization_dan_scopes (Eager loading with() dan Query Scopes)
✓ test_database_transaction_dan_rollback (Atomisitas & Auto-rollback)
✓ test_soft_deletes_dan_restore (Audit trail soft delete & restore)
✓ test_domain_event_dispatched (Event BookBorrowed dipancarkan)
✓ test_value_objects_isbn_dan_denda (Validasi ISBN & kalkulasi denda)

Tests: 6 passed (38 assertions) | Execution Duration: 1.05s
```

BAB V: KESIMPULAN

Berdasarkan seluruh tahapan perancangan, implementasi, dan pengujian yang telah diselesaikan pada Sistem Perpustakaan Modular berbasis Laravel 13, dapat ditarik beberapa kesimpulan pokok:

1. **Modularitas & Decoupling:** Penerapan Modular Monolith dengan Domain-Driven Design (DDD) berhasil memecah kerumitan logika sistem ke dalam Bounded Context yang jelas (Catalog, Membership, Circulation, Shared Kernel). Ketergantungan langsung antar tabel berhasil dihilangkan melalui perantara Contract (Interface) dan Domain Event.
2. **Fleksibilitas Relasi Eloquent:** Relasi Many-to-Many dengan pivot custom (added_at) dan Self-referencing hierarkis terbukti sangat ampuh memodelkan hubungan entitas dunia nyata, sedangkan Snapshot Pattern pada detail transaksi menjamin keaslian data historis.
3. **Efisiensi Query yang Terukur:** Penggunaan Eager Loading terbukti memangkas jumlah eksekusi query dari 11 query menjadi 3 query konstan, mengeliminasi risiko kelambatan sistem akibat masalah N+1 Query.
4. **Konsistensi Data Transaksional:** Pemanfaatan Database Transaction (DB::transaction) menjamin integritas data secara atomik, di mana kegagalan bisnis pada satu komponen secara otomatis membatalkan seluruh operasi tanpa anomali data.
5. **Integritas Domain melalui Value Objects:** Pemakaian Value Object (ISBN dan Denda) memperkuat domain model aplikasi dengan memastikan data selalu valid sejak awal pembuatan objek.

LAMPIRAN: TAUTAN REPOSITORY GITHUB

Seluruh kode sumber, riwayat commit Git, modul, dan script pengujian dapat diakses pada repositori publik berikut:

🔗 https://github.com/panjiprasetyaa/praktikum-backend-6.git